架构辩论赛记录

成员A

prompt

我们是一个四个人的学生小组，十周之内想做一个约拍服务平台，现在很多年轻人（包括普通想发朋友圈的人和自己做自媒体需要在社交平台发图的人）日常，周末这些时间都有出片的需求（现在有旅拍产业但是仅限于景点产业）；日常里摄影师和化妆师这些没有固定渠道去联系，现在线上很多约拍需求分散在微信群、小红书，我们想把这个做一个业务闭环
用户可以发布自己的约拍需求，摄影师妆造师提供服务，用户发布约拍需求或浏览摄影师主页，双方沟通后确认价格地点时长等发起预约，进入确认订单、进行中、完成、评价的流程
平台提供作品集展示、档期管理、风格标签、信用评价、站内通知，以及后台审核等功能，你推荐我们用什么架构模式?

回复

十周时间要从零跑通一个包含双端（用户+服务者）、有交易流、图片内容和排班机制的业务闭环，时间可以说是**非常紧张且极具挑战性的**。

基于这个极其明确的“十周交付”死命令，我强烈建议你们采用 **分层架构（Layered Architecture）为核心，配合 MVC 模式** 来组织代码。

同时，我必须非常直白地给出一条现实建议：**在十周的 V1 版本中，绝对、绝对不要碰微服务（Microservices）**。

以下是具体的架构组合策略和详细阐释：

1. 核心底座：分层架构 (Layered Architecture) + MVC

这是目前最成熟、开发效率最高的主流模式。它的核心思想是“职责分离”，让团队成员能快速并行开发，不用把时间浪费在复杂的系统调用上。

在这个约拍平台中，你们的系统应该严格划分为以下几层：

- **表现层 / 接口层 (Controller - MVC 中的 C)：**
  - **职责：** 只负责接收前端（小程序/后台）的 HTTP 请求，校验参数（比如检查传入的手机号格式），然后立刻把任务丢给下一层。
  - **注意：** 绝对不要在这里写业务逻辑！不要在 Controller 里写 SQL 或判断定金比例。
- **业务逻辑层 (Service - 分层架构的核心)：**
  - **职责：** 平台的心脏。所有的核心规则都在这里实现。例如：检查摄影师档期是否冲突、计算这单该收多少手续费、流转订单状态（从“待接单”到“已确认”）。
- **数据访问层 (DAO/Repository - MVC 中的 M 衍生)：**
  - **职责：** 只跟数据库（MySQL）和缓存（Redis）打交道，负责把业务层算好的数据存进去，或者按要求把作品集的数据取出来。

- **视图层 (View - MVC 中的 V)：**
  - 在这个前后端分离的时代，你们的 View 就是前端的小程序和 Vue/React 管理后台界面。

为什么选它？ 十周项目最大的敌人是“沟通成本”和“调试成本”。分层架构所有代码都在一个进程里，报错了看控制台一眼就能定位，能最大化保障你们如期交付。

2. 为什么坚决劝退 微服务 (Microservices)？

很多技术团队喜欢一开始就上微服务，觉得高级，但在十周的时限下，这往往是灾难。

- **基建耗时：** 微服务需要搞定服务注册、网关、负载均衡、自动化部署等一堆基建。十周时间，可能前两周你们都在配环境。
- **分布式事务的噩梦：** 约拍平台有典型的强事务场景。比如：用户取消订单 -> 触发退定金 -> 触发释放摄影师档期。在分层架构里，加个 `@Transactional` 注解，两行代码搞定，一旦出错全部回滚。在微服务里，你需要写复杂的分布式事务补偿逻辑，极容易出 Bug 导致资金对不上。
- **结论：** 微服务是用来解决“组织扩大”和“海量并发”问题的，不是用来解决“从0到1跑通业务”的。等你们拿到下一轮融资，单日订单破万时，再把庞大的分层架构拆成微服务也不迟。

3. 巧妙融入 事件驱动 (Event-Driven) 作为局部优化

虽然不建议做全盘的事件驱动架构（那同样太复杂），但在**单体分层架构内部，我强烈建议局部使用事件驱动模式**（例如使用 Spring 框架自带的 `ApplicationEvent`）。

这能极大地保持代码干净，非常适合约拍平台的“副作用”逻辑。

- **场景举例：** 当订单状态变为“已完成”时。
- **糟糕的写法（全塞在 Service 里）：** 修改订单状态 -> 写发微信通知的代码 -> 写给摄影师加信用分的逻辑 -> 写更新财务报表的逻辑。代码越来越臃肿，且发微信如果超时了，会导致整个接口卡住。
- **事件驱动的写法：** 核心订单 Service 只负责修改状态，然后大喊一声（发布事件）：`"订单 10086 已完成！"`。
- **监听者：**
  - 通知模块监听到后，默默去发微信模板消息。
  - 评价模块监听到后，自动给用户发一条“快去评价吧”的站内信。
  - 信用分模块监听到后，去给摄影师加分。
- **优势：** 你的核心交易流程非常轻快稳定，且后期增加新功能（比如完成后送优惠券）完全不需要改老代码，只需加一个监听器即可。

总结来说：**用 MVC 分配请求，用严格的分层架构沉淀业务逻辑，用内部事件驱动解耦附属功能，把微服务无限期延后。** 这是一个能让你们在十周内最稳妥上线的实战策略。

成员B

prompt

我们是一支智能软件工程专业的本科生团队，想做一个约拍服务平台。现在很多年轻人（包括普通想发朋友圈的人和自己做自媒体需要在社交平台发图的人）日常，周末这些时间都有出片的需求（现在有旅拍产业但是仅限于景点产业）；日常里摄影师和化妆师这些没有固定渠道去联系，现在线上很多约拍需求分散在微信群、小红书，我们想把这个做一个业务闭环。

用户可以发布自己的约拍需求，摄影师妆造师提供服务，用户发布约拍需求或浏览摄影师主页，双方沟通后确认价格地点时长等发起预约，进入确认订单、进行中、完成、评价的流程。平台提供作品集展示、档期管理、风格标签、信用评价、站内通知，以及后台审核等功能。

目前正处于架构设计阶段，以下是我们的项目真实约束与详细痛点：

【项目与工程约束】

1. 极限开发周期：课程大项目，要求 10 周内完成核心业务闭环的开发与上线。
2. 团队背景与能力：团队成员具备扎实的计算机专业基础（数据库、操作系统、算法等），但缺乏大型分布式系统架构、生产环境排错以及服务器运维实战经验。
3. 终端形态：移动端独立软件/App。

【核心业务与技术特征】

1. 复杂状态机流转：采用担保交易模式，订单生命周期涉及多方状态流转与制约，涵盖：前置沟通、定金/全款托管、线下拍摄、限时图片交付、7天超时自动确认收货、纠纷仲裁、双向信用评价等。
2. 重文件 I/O 与带宽压力：业务高度依赖大量高分辨率原片与精修片的上传、在线预览与打包下载，对系统的网络带宽和存储调度要求极高。
3. 强 IM 业务耦合：即时通讯不仅是文字沟通，需在 1v1 聊天信息流中强绑定核心交易状态，例如在聊天中发送示例图片后，直接触发并生成/确认“报价单”流转。

【核心请教的架构问题】
基于 10 周的死命令和我们当前的能力边界，你推荐我们采用什么样的整体架构（例如MVC、分层、微服务、事件驱动）？如何在保证项目能平稳落地的同时，在代码结构和模块划分上体现出足够的“架构设计深度”，避免后期代码腐化？
请详细一些，考虑到我们只是普通大学生，不要太过专业术语。
请你告诉我们用什么架构设计比较好（参考有课上讲到的MVC，分层，微服务和事件驱动或者结合）。

回复

核心结论
在 10 周开发周期且缺乏服务器运维经验的前提下，绝对不适合采用“微服务架构”（服务拆分、

部署、网络通信、分布式事务、监控等会严重拖垮项目进度）。但针对本项目复杂的业务闭环，简单的 CRUD 无法满足需求。

最优架构选型：分层架构（主） + 模块化单体（核心） + 局部事件驱动（点缀） + MVC（表现层模式）

1. 总体架构推荐

我们需要一个工程上可控，且在代码划分上体现出架构深度的组合方案。

模块化单体（Modular Monolith）
保持一个完整的后端项目（不进行服务物理拆分），但内部按业务领域进行严格的模块化拆分。每个模块“像微服务一样设计”，但不独立部署。

业务模块拆分建议：

- 用户模块（User）
- 约拍需求模块（Demand）
- 服务提供者模块（Photographer / Stylist）
- 订单模块（Order）：最核心模块
- 聊天模块（IM）
- 文件模块（File）
- 评价与信用模块（Review）
- 后台审核模块（Admin）

架构优势：规避了分布式系统的运维复杂度，但在结构上具备“微服务思维”，能够有效防止代码腐化。

经典分层架构（必须严格执行）
为了体现架构深度，必须放弃仅有 Controller + Service 的初级写法，采用经典 4 层架构，将复杂业务（如订单流转）下沉至核心层：

- Controller（接口层）：负责请求接收与参数校验。
- Service（业务调度层）：负责协调各个领域模型完成具体用例。
- Domain（领域模型层）：最重点部分，封装核心业务逻辑与状态流转。
- Repository（数据访问层）：纯粹的数据库 CRUD 操作。

MVC（表现层模式）
仅用于接口层与视图层的数据交互规范：

- Controller：处理 HTTP 请求。
- Service：调度业务逻辑。
- Model：传递数据对象（DTO/VO）。

局部事件驱动（关键解耦点）
在核心业务节点引入事件机制，避免代码重度耦合，极大提升系统的扩展性。

- 场景 1：订单状态变化多方通知
  - 传统写法：充斥大量 if-else。
  - 事件驱动：状态变更后抛出 OrderPaidEvent、OrderCompletedEvent、OrderTimeoutEvent。辅助功能（如通知摄影师、开启评价）通过监听事件异步执行。
- 场景 2：IM 与交易的耦合
  - 业务需求：聊天中点击“生成报价单”。
  - 事件驱动：IM 模块发送特定事件，Order 模块监听并触发“创建报价单”逻辑，实现 IM 与核心交易的物理隔离。
  - 技术落地：使用 Spring Event（Java 生态）或自定义简单的 EventBus。

2. 核心难点设计

订单系统（最关键）
订单不是简单的数据库更新，而是一个严谨的生命周期状态机。必须使用显式状态机进行管理。

状态流转链路：
INIT（沟通中） -> QUOTED（已报价） -> CONFIRMED（已确认） -> PAID（已支付） -> IN_PROGRESS（拍摄中） -> DELIVERING（交付中） -> COMPLETED（完成） -> REVIEWED（已评价）

代码规约建议：
严禁在 Service 层使用零散的 if 判断。应将状态与行为绑定至领域模型中：

```
class Order {
    OrderStatus status;

    void pay() {
        if (status != OrderStatus.CONFIRMED) {
            throw new IllegalStateException("当前状态无法进行支付操作");
        }
        status = OrderStatus.PAID;
    }
}
```

总结

这是一个基于分层架构的模块化单体系统，在订单等核心业务中引入了领域模型和状态机确保逻辑严谨，并在 IM 与订单的边界解耦处巧妙使用了事件驱动机制。

成员C

prompt:

我们正在做一个课程项目,项目是“约拍服务平台”。 项目背景: 现在很多年轻人,包括普通想发朋友圈的人、自媒体创作者、周末日常出片的人,都有约拍需求。但目前这些需求分散在微信群、小红书等平台,摄影师和化妆师也缺少统一渠道去展示作品、管理档期、接单和沟通。 项目目标: 我们希望做一个业务闭环平台。用户可以发布约拍需求,也可以浏览摄影师主页;摄影师/化妆师可以提供服务、展示作品集、设置档期、管理订单;双方沟通后确认价格、地点、时间,并发起预约,订单会经历“确认订单、进行中、完成、评价”等流程。 核心功能包括:

1. 用户注册登录
2. 用户发布约拍需求
3. 摄影师/化妆师主页展示
4. 作品集展示
5. 档期管理
6. 风格标签
7. 信用评价
8. 站内通知
9. 后台审核
10. 订单状态流转

团队约束: 这是一个大二课程项目,开发周期约 10 周,团队人数4人,主要目标是完成可运行的MVP,而不是做大规模商业系统。团队需要考虑开发复杂度、可维护性、可扩展性、性能、学习成本和 10 周内可行性。 请对比以下四种架构方案:

1. 纯 MVC 架构
2. 纯分层架构
3. 微服务架构
4. MVC + 分层架构

请从以下维度进行分别说明:

1. 每种架构适合本项目的地方
2. 每种架构不适合本项目的地方
3. 最终推荐哪一种架构
4. 为什么不推荐另外三种架构
5. 如果选择推荐架构,应该如何划分主要模块

回复：各方案适合与不适合本项目之处

纯 MVC 架构

适合的地方：框架自带 MVC 骨架（Spring MVC、Django、Rails），团队几乎零配置就能跑起来，非常适合快速做出原型。10 个核心功能都可以在 Controller 里直接堆砌，短期内进展看得见。

不适合的地方：约拍平台的订单状态流转（确认→进行中→完成→评价）、档期校验、评价计算等逻辑，如果全塞进 Controller 会迅速变成"胖 Controller"。到第 7-8 周加评价或通知功能

时，代码耦合度会让改动变成雷区，测试也几乎无法单独覆盖业务逻辑。

纯分层架构

适合的地方：强制将表现层、业务层、数据访问层分开，职责清晰，UserService、OrderService、ScheduleService 各司其职，后期新增功能时修改范围可控。

不适合的地方：如果团队没有经验，从零开始设计一套严格的分层规范（含接口抽象、依赖倒置等）本身就需要较多时间对齐，而且脱离 MVC 框架骨架后，路由、参数绑定、异常处理等"框架送的福利"都要自己补，对大二团队有一定认知负担。

微服务架构

适合的地方：理论上把用户服务、订单服务、消息通知服务拆开，各自独立演进，扩展性极强。如果这是一个有 3 年周期的商业项目，这套架构非常合理。

不适合的地方：微服务要求团队额外处理服务注册与发现、API 网关、服务间鉴权、分布式事务、容器化部署等基础设施问题——这些与业务功能本身毫无关系，却会消耗绝大多数开发时间。10 周内，4 个人大概率连基础设施都搭不完，更别说实现业务闭环。这是典型的过度设计（Over-engineering），不推荐。

MVC + 分层架构（推荐方案）

适合的地方：以成熟 MVC 框架为骨架（Spring Boot / Django REST Framework 等），路由、参数绑定、安全等"体力活"由框架包办；在框架内部主动遵循 Controller → Service → Repository → Model 四层约定。业务逻辑集中在 Service 层，既能快速迭代，又避免后期因耦合带来的技术债。订单流转、档期冲突检测、评价聚合等复杂业务都有明确的"家"——Service 层，不会散落在各处。

不适合的地方：相比纯 MVC，需要提前约定分层规范并执行代码审查，有小幅额外纪律成本。但这个成本在第 1-2 周定好框架后基本可以忽略。

为什么不推荐另外三种

纯 MVC 能完成 MVP，但会在后期留下大量技术债，订单流转、档期管理这类多步骤业务会把 Controller 撑爆，最后两周改 bug 的时间会远超预期。

纯分层架构在概念上没问题，但它要求团队自行搭建许多 MVC 框架本来就提供的基础设施，性价比不如在 MVC 上叠加分层。

微服务架构直接排除——10 周 4 人无法在完成业务功能的同时维护一套分布式基础设施，MVP 交付风险极高。

如果选择 MVC + 分层架构，主要模块如何划分

```
约拍平台（单体应用）
│
├── Controller 层（HTTP 入口）
│   ├── UserController        注册、登录、个人信息
│   ├── ServiceProviderController   摄影师/化妆师主页、作品集
│   ├── NeedController        用户发布约拍需求
│   ├── OrderController       下单、状态流转
│   ├── ScheduleController    档期查询与管理
│   ├── ReviewController      评价提交与展示
│   └── NotificationController  站内通知
│
├── Service 层（业务逻辑核心）
│   ├── UserService           认证、积分/信用计算
│   ├── OrderService          订单状态机（确认→进行中→完成→评价）
│   ├── ScheduleService       档期冲突检测
│   ├── ReviewService         评分聚合
│   ├── NotificationService   通知触发与入队
│   └── AuditService          后台审核（作品集、注册审核）
│
├── Repository 层（数据访问）
│   ├── UserRepository
│   ├── OrderRepository
│   ├── ScheduleRepository
│   ├── PortfolioRepository
│   └── ReviewRepository
│
└── Model 层（领域实体）
    ├── User / ServiceProvider
    ├── Order（含状态枚举）
    ├── Schedule
    ├── Portfolio / StyleTag
    └── Review / Notification
```

几个关键设计建议：订单状态流转建议用状态机模式（一个枚举 + 一个 `transition()` 方法），避免 `if/else` 遍地；档期冲突检测逻辑集中在 `ScheduleService`，不要在 Controller 里做；通知可以先用同步调用，后期有精力再换消息队列，不影响上层接口。

成员D

prompt

你现在是一位有多年经验的资深软件架构师，请针对我们的约拍服务平台进行系统性的架构分析。

项目背景：我们要做一个约拍服务平台，核心业务是用户发布约拍需求，摄影师/化妆师提供服务，双方沟通确认后走订单流程（预约→确认→进行中→完成→评价）。平台还涉及作品集展示、档期管理、风格标签、信用评价、站内通知、后台审核等功能。

项目约束：4人团队、10周周期、课程项目、本科生技术水平。

请从以下角度给出你的专业架构建议：

1. 推荐什么架构方案，为什么
2. 为什么不选其他方案（纯MVC、微服务、纯分层）
3. 具体的模块划分建议
4. 需要关注的架构风险
5. 技术选型建议

回复

一、项目约束理解

约拍服务平台的核心业务是：用户发布约拍需求，摄影师/化妆师提供服务，双方沟通确认后走订单流程（预约 → 确认 → 进行中 → 完成 → 评价）。平台还涉及作品集展示、档期管理、风格标签、信用评价、站内通知、后台审核等功能。

关键约束条件：4 人团队、10 周周期、课程项目、团队大概率是本科生技术栈水平。

二、架构方案推荐：MVC + 分层架构

作为架构师，明确推荐 MVC + 分层架构（Layered Architecture）方案。以下从多个维度给出专业判断。

2.1 为什么选 MVC + 分层

约拍平台的核心是一个典型的"交易撮合"型业务系统。它的业务流程是线性且清晰的：发布需求 → 浏览匹配 → 沟通确认 → 下单 → 履约 → 评价。这种业务形态天然适合分层架构，因为每一层的职责非常明确：

层次	职责说明
表现层（Controller / MVC）	处理 HTTP 请求、参数校验、返回响应。前后端分离的情况下，这一层就是 REST API 的入口。
业务逻辑层（Service）	最关键的一层。订单状态机（从"待确认"到"已完成"的流转）、信用分计算规则、档期冲突检测——核心逻辑全部收敛在此层。
数据访问层（DAO/Repository）	封装数据库操作，让上层不关心具体的 SQL 或 ORM 细节。

层次	职责说明
数据模型层（Entity/Model）	定义用户、订单、作品、评价等核心领域对象。

2.2 为什么不选其他方案

纯 MVC： 对于简单的 CRUD 项目够用，但约拍平台的订单流程有明确的状态机，信用评价有计算规则，如果把这些逻辑全塞进 Controller，到中后期 Controller 会变得臃肿不堪。实际项目中经常出现一个 OrderController 写到上千行，谁都不敢改的情况。

微服务架构： 这是"用大炮打蚊子"。4 个人 10 周，光是搭建服务注册发现、API 网关、分布式配置中心就要花掉 2-3 周。而且微服务的调试成本非常高，一个请求跨 3 个服务出了问题，排查链路会很痛苦。校园级用户量和 4 人团队规模完全不需要微服务带来的水平扩展能力。

纯分层（无 MVC）： 分层本身只解决纵向分层的问题，但不解决请求如何路由、视图如何渲染的问题。在使用 Spring Boot 这类框架时，MVC 模式天然嵌入在表现层，两者是互补而非替代的关系。

2.3 具体的分层模块建议

针对约拍平台，建议如下模块划分：

模块目录	子模块	说明
controller/	UserController、OrderController、PortfolioController、ScheduleController、ReviewController	表现层：REST API 接口（作品集、档期、评价相关）
service/	UserService、OrderService、MatchService、CreditService、NotificationService	业务逻辑层（核心：订单状态机、匹配推荐、信用分计算、通知）
repository/	—	数据访问层
model/	entity/、dto/、vo/	实体模型（数据库实体、数据传输对象、视图对象）
common/	—	通用工具、异常处理、配置

三、需要特别关注的架构风险

风险项	说明	建议措施
订单状态机复杂度	约拍订单不是简单的"下单→完成"，中间涉及沟通确认、档期协调、可能的取消和改期。	用状态模式或枚举状态机管理，不要用 if-else 堆砌。

风险项	说明	建议措施
图片存储	作品集展示意味着大量图片，存数据库会导致严重性能问题。	使用对象存储（阿里云 OSS 或 MinIO），数据库只存 URL。
并发问题	同一摄影师的同一时间段可能被多人同时预约。	在 Service 层做好档期的并发控制，可用乐观锁或分布式锁。

四、技术选型建议

从 10 周可行性和学习成本考虑，建议以下技术栈：

层次	建议选择	选择理由
后端框架	Spring Boot	生态成熟，MVC+分层天然支持，学习资料丰富
前端框架	Vue 3 或 React	根据团队熟悉度选择，两者均可
数据库	MySQL	稳定可靠，订单类业务需要事务支持
图片存储	阿里云 OSS / MinIO	作品集图片量大，不应存数据库
缓存（可选）	Redis	热门摄影师列表、档期查询可缓存
部署方式	Docker + 单机部署	课程项目够用，不需要 K8s

五、总结

MVC + 分层架构对约拍服务平台来说是"刚好够用"的选择——既不会因为过于简单导致后期代码混乱，也不会因为过度设计浪费有限的开发时间。

架构设计的核心原则：在约束条件下做最合适的选择，而不是最"高级"的选择。

具体优势总结：

1. Controller 很薄，只做"翻译"工作——把 HTTP 请求翻译成 Service 调用，把 Service 的返回翻译成 HTTP 响应。
2. Service 层扛起所有业务逻辑，订单流转和档期管理等复杂逻辑全收敛于此，一个人改订单逻辑不会影响另一个人写接口。
3. 前后端完全分离，后端只提供 REST API，前端独立开发，4 个人可以 2+2 或 1+3 分工。
4. 未来可平滑升级，Service 层天然就是微服务拆分的边界——把 OrderService 连同 Repository 提出去就是独立的订单微服务。

"架构辩论赛"结论

一、 最终结论

项目的最终架构模式选择为：**MVC + 分层架构** 为主体，内部采用 **模块化单体** 组织代码，并在关键业务节点引入 **轻量事件驱动** 作为补充。

> 这是在 10 周开发周期、4 人团队条件下，能够稳定落地且保证代码结构清晰的**最优解**。

二、 为什么选这个方案?

1. **工程上绝对可控**
   - MVC 框架负责接口、路由、参数处理等基础能力，团队不需要重复造轮子。
   - 分层架构保证了职责清晰：Controller 只处理请求，Service 承载业务逻辑，Repository 负责数据持久化。
2. **能承载核心业务复杂性**
   - 约拍平台的本质不是简单的 CRUD，而是复杂的订单状态机、档期冲突校验与信用评价体系。
   - 严格分层后，这些复杂的逻辑会集中在 Service 或 Domain 层，不会散落。
3. **高效支持并行开发**
   - 4 人团队可以按业务模块拆分（如用户、订单、档期、评价等），互不干扰。
   - 单体应用避免了微服务之间繁琐的网络通信与联调成本。
4. **具备“可进化性”**
   - 模块化单体让代码结构在思想上接近微服务。
   - 未来如果要拆分独立的服务，完全可以按现有的模块边界进行平滑拆分。
5. **局部事件驱动解决解耦问题**
   - 订单完成、系统通知、评价开启、信用分变动等“副作用”统一通过事件触发。
   - 有效避免了核心主流程代码的臃肿，极大提升了系统的扩展性。

三、 为什么不选其他方案?

- **纯 MVC（不选）：**
  短期开发确实快，但所有的业务逻辑会堆积在 Controller 层。订单流转、档期校验、评价逻辑等会迅速失控，后期维护成本极高，极其容易崩盘。
- **纯分层架构（不单独选）：**
  它仅仅是一种结构思想，不提供 Web 层的基础能力。如果脱离了 MVC 框架，团队需要自己处理路由、参数绑定、异常处理等底层逻辑，对当前团队来说性价比极低。
- **微服务架构（明确不选）：**
  引入微服务需要解决服务注册、API 网关、跨服务通信、分布式事务、容器化部署等问题。这些运维和中间件成本远高于业务开发本身，在 10 周内极大概率无法完成业务闭环。

四、 最终架构形态总结

> **一句话总结：**
> 这是一个基于 MVC 框架的模块化单体系统，内部采用严格分层来承载复杂业务，并在关键业务节点用事件机制进行解耦。

结构本质：

- **外层** 是 MVC 负责接口映射与交互。
- **中间** 是分层架构稳稳承载业务。
- **内部** 是模块化划分领域边界。
- **关键点** 用事件驱动降低耦合。

方案的核心价值在于：
不盲目追求“高级架构”，而是在团队现有的时间限制和能力边界内，做到**结构清晰、代码可维护、且能按时交付**。